

python基础

基础语法

导入模块

- `import <module>`: 导入整个模块,使用时需通过模块名访问模块中的对象,可能会与当前命名空间中的其他对象发生命名冲突
- `from <module> import <name>`: 模块中导入特定的对象 (函数、类、变量等),使用时直接引用导入的对象,不需模块名前缀
- `from <module> import *`: 导入所有对象, 尽量避免使用, 可能导致命名冲突和代码混淆
:::tip `if __name__ == '__main__':` 是什么?
`__name__` 是一个特殊变量, 表示当前模块的名字
- 当模块被直接运行时, `__name__` 的值赋为 `'__main__'`
- 当模块被导入时, `__name__` 的值为模块的实际名字
因此, 如果模块被**直接运行**, 就执行下面的代码, 如果**被导入**, 只提供函数和类
...

流程控制

Python 中不用大括号来显式地分块, 而是用冒号配合缩进 (Indent)。代码块与代码块之间至少隔着一个空行表示结束。当一个代码块必须存在, 又不想写任何语句的时候, 可以写一个 `pass` 作为占位符。

- `break`: 跳出当前循环
- `continue`: 跳过当前循环的剩余部分, 进入下一次循环

条件分支

```
if condition:
    pass
elif another_condition:
    pass
else:
    pass

x if condition else y # 三目运算符, 条件为真返回 x, 否则返回 y
```

如果有多个条件, 用 `and` 和 `or` 逻辑运算符连接

for循环

```
for item in iterable:
    pass

# 带索引的 for 循环
for i, item in enumerate(iterable):
    pass
```

while循环

```
while condition:
    pass
```

异常控制

- try-except 语句用于捕获和处理异常，避免程序崩溃

```
# 主动抛出异常，Exception 是所有内置异常的基类，可以替换为更具体的异常类
raise Exception("错误信息")

try:
    # 可能会引发异常的代码块
except (ExceptionType1, ExceptionType2):
    # 处理特定类型的异常，这里需要填入异常类名
except Exception as e:
    # 处理未知异常，Exception 是大多数异常类的父类
    print("发生了一个未知错误:", e)
else:
    # 当没有发生任何异常时执行的代码，例如打印成功信息
finally:
    # 无论是否发生异常都会执行的代码，例如文件关闭
```

- with 语句用于简化资源管理，确保资源正确释放，例如文件操作

```
# 自动处理清理逻辑，无需手写 finally:
with open("data.txt", "r") as f:
    content = f.read()
# 离开 with 块，文件自动关闭，即使出错也一样

# 等价于:
f = open("data.txt", "r")
try:
    content = f.read()
finally:
    f.close()
```

数据类型

Python 中的变量不需要声明。每个变量在使用前都必须赋值，变量赋值以后该变量才会被创建。在 Python 中，变量没有类型，数据有类型，我们所说的**类型**是变量所指的内存中**对象的类型**。等号 = 用来给变量赋值。等号左边是变量名，右边是存储在变量中的值。

标准数据类型

Python3 中有 6 种标准数据类型，以及 bool 布尔类型（int 子类），分为两类

- 不可变数据（4 个）：Number（数字）、String（字符串）、bool（布尔）、Tuple（元组）
- 可变数据（3 个）：List（列表）、Dictionary（字典）、Set（集合）

运算符

- 算术运算符：包括 + - * / 加减乘除，// % ** 整除、取余、幂运算
- 比较运算符：和位运算符和C/C++相同
- 赋值运算符：= += -= *= /= %=，注意 Python 中没有 ++ -- 的运算符，只能通过 += 1 实现
- 逻辑运算符：and or not，相当于C中 && || !，具有短路机制
- 三目运算符：a if cond else b，相当于C中的 cond ? a : b
- 特殊条件运算符：

1. in not in
2. is is not

Number(数字)

- Python3 支持 int (长整型)、float、bool、complex (复数)
- 复数由实数部分和虚数部分构成，可以用 a + bj 或者 complex(a, b) 表示，复数的实部 a 和虚部 b 都是浮点型
- 内置的 type() 函数可以用来查询变量所指的对象类型
- isinstance()也可判断对象类型，isinstance()会认为子类是一种父类类型。

:::tip

Python3 中，bool 是 int 的子类，True 和 False 可以和数字相加，True1、False0 会返回 True。

:::

```
a, b, c, d = 20, 5.5, True, 4+3j
print(type(a), type(b), type(c), type(d))
# <class 'int'> <class 'float'> <class 'bool'> <class 'complex'>
print(isinstance(c, bool)) # True
print(isinstance(c, int)) # True
```

String(字符串)

- 使用引号(' 或 ")来创建字符串，不可变，赋值修改本质上是重新生成另一个字符串
- 基本操作
 1. 拼接：+，重复：*，切片：[start:end:step] (左闭右开)
 2. in not in：是否包含
- 常用方法

```
s = "Hello, Python!"

s.find("Python")      # 7, 找不到返回 -1
s.index("Python")     # 7, 与find一样，但找不到抛出异常
s.count(",")          # 2, 统计出现次数
s.startswith("Hello") # True
s.endswith("!")       # True
s.strip()             # 'Hello, Python!' 去两端的空白字符
s.split(",")          # ['Hello', ' Python!'], 默认分割次数为 -1, 即分割所有
s.split(",", 1)       # ['Hello', ' Python!'] ← 限制分割次数
s.upper()             # 'HELLO, PYTHON!'
s.lower()             # 'hello, python!'
```

```
s.replace("Python", "world") # 'Hello, world!', 替换字符串
"-".join(["a", "b", "c"]) # 'a-b-c'
" ".join(["Hello", "world"]) # 'Hello world'
```

- 字符串格式化

```
name = "Alice"
age = 30
score = 95.5
print(f"我叫{name}, 今年{age}岁, 成绩是{score:08.3f}分") # f-string, Python 3.6+
print("我叫{}, 今年{}岁, 成绩是{:08.3f}分".format(name, age, score)) # str.format(), Python 2.7+
print("我叫%s, 今年%d岁, 成绩是%08.3f分" % (name, age, score)) # % 格式化, 较旧的方式
```

List(列表)

- 列表是 Python 中最常用的数据结构之一，是一个有序的可变集合，用方括号 `[]` 定义，元素之间用逗号分隔
- 基本操作
 1. 创建: `lst=[]` 或者 `lst=list()`，列表元素可以是任意类型，甚至是另一个列表（嵌套列表）
 2. 索引和切片: `lst[index]`，切片: `lst[start:end:step]`，支持负数索引，`lst[-1]` 表示最后一个元素
 3. 添加: `lst.append(x)` 在末尾添加元素，`lst.insert(i, x)` 在指定位置插入元素，`lst.extend(iterable)` 扩展列表
 4. 删除: `lst.remove(x)` 删除第一个匹配的元素，`lst.pop(i)` 删除指定位置的元素并返回(默认末尾)，`del lst[i]` 删除指定位置的元素
 5. 查找: `x in lst` 检查元素是否存在，`lst.index(x)` 返回第一个匹配的索引，找不到抛出异常
 6. 排序: `lst.sort()` 原地排序，`sorted(lst)` 返回排序后的新列表
 7. 其他: `len(lst)` 返回列表长度，`lst.count(x)` 统计元素出现次数，`lst.reverse()` 原地反转列表
- 列表推导式: `[表达式 for 变量 in 可迭代对象 if 条件]`，用于生成新的列表

```
[x**2 for x in range(10) if x % 2 == 0] # [0, 4, 16, 36, 64]
# 等价于
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x**2)

matrix = [[1,2,3],[4,5,6],[7,8,9]]
[x for row in matrix for x in row] # [1, 2, 3, 4, 5, 6, 7, 8, 9]
# 等价于
flattened = []
for row in matrix:
    for x in row:
        flattened.append(x)
```

- 常见用法

```
# 1. 列表解包
a, b, c = [1, 2, 3]
first, *rest = [1, 2, 3, 4, 5] # first=1, rest=[2,3,4,5]
*head, last = [1, 2, 3, 4, 5] # head=[1,2,3,4], last=5

# 2. zip 将多个可迭代对象打包成一个元组列表, 如果长度不一致, zip 会以最短的可迭代对象为准
names = ["Alice", "Bob"]
scores = [90, 85]
list(zip(names, scores)) # [('Alice', 90), ('Bob', 85)]
for name, score in zip(names, scores): # 同时遍历多个列表
    print(f"{name} 考了 {score} 分")

# 3. enumerate 带索引遍历
for i, val in enumerate(["a", "b", "c"]):
    print(i, val)
```

Tuple(元组)

- 元组是一个有序的不可变集合, 用圆括号 `()` 定义, 元素之间用逗号分隔
- 基本操作
 - 创建: `tup=()` 或 `tup=(1, 2, 3)` 或 `tup=1, 2, 3`, 单元素元组需要在元素后加逗号, 如 `tup=(1,)`
 - 元组不可变, 不能修改元素, 但可以包含可变对象, 如列表, 可以修改列表内部的元素

```
tup = (1, [2, 3], 4)
tup[1][0] = 20 # 修改列表内部的元素, tup变为(1, [20, 3], 4)
```

- 索引和切片: 与列表相同, 支持负数索引
 - 查找: `x in tup` 检查元素是否存在, `tup.index(x)` 返回第一个匹配的索引, 找不到抛出异常
 - 其他: `len(tup)` 返回元组长度, `tup.count(x)` 统计元素出现次数
- 常见用法

函数的多个返回值: Python 函数可以返回多个值, 实际上是返回一个元组

```
def get_coordinates():
    return 10, 20 # 返回一个元组 (10, 20)
x, y = get_coordinates() # 元组解包, x=10, y=20

# 函数定义中使用 *args 和 **kwargs 来接受可变数量的位置参数和关键字参数
def func(*args, **kwargs):
    print("位置参数:", args) # args 是一个元组 (1, 2, 3)
    print("关键字参数:", kwargs) # kwargs 是一个字典 {'a': 4, 'b': 5}
func(1, 2, 3, a=4, b=5)

# 交换变量值: 利用元组的不可变特性, 可以同时交换多个变量的值, 而不需要临时变量
a = 1
b = 2
a, b = b, a # 从右往左执行, 先创建一个元组 (b, a), 然后将其解包赋值给 a 和 b, 最终 a=2, b=1
```

Dictionary(字典)

- 字典是一个无序的可变集合，用花括号 `{}` 定义，包含键值对，键和值之间用冒号 `:` 分隔，键值对之间用逗号分隔
- 基本操作
 - 创建：键必须是不可变类型（如字符串、数字、元组），值可以是任意类型

```
d = {}  
d = dict() # 使用 dict() 构造函数创建空字典  
d = {"name": "Alice", "age": 30} # 使用字面量创建字典  
d = dict(name="Alice", age=30) # 使用关键字参数创建字典  
d = dict([("name", "Alice"), ("age", 30)]) # 使用可迭代对象创建字典
```

- 访问和修改：`d[key]` 访问值，`d[key] = value` 修改或添加键值对，`d.get(key, default)` 安全访问，找不到返回默认值
- 删除：`del d[key]` 删除键值对，`d.pop(key, default)` 删除并返回值，没有默认返回值则抛出异常，`d.clear()` 清空字典
- 遍历

```
d = {"name": "Alice", "age": 25, "city": "Beijing"}  
  
# 遍历键（默认）  
for key in d:  
    print(key) # name / age / city  
  
# 遍历值  
for val in d.values():  
    print(val) # Alice / 25 / Beijing  
  
# 遍历键值对（最常用）  
for key, val in d.items():  
    print(f"{key}: {val}") # name: Alice ...  
  
# 带索引遍历  
for i, (k, v) in enumerate(d.items()):  
    print(i, k, v) # 0 name Alice ...
```

- `d.update(other_dict)` 将另一个字典的键值对更新到当前字典，存在同名键则覆盖
- 字典推导式：`{表达式 for 变量 in 可迭代对象 if 条件}`，用于生成新的字典
 - 常见用法

```
# 1. defaultdict: 访问不存在的键时自动初始化默认值  
from collections import defaultdict  
dd = defaultdict(int) # 默认值为0  
for ch in "banana":  
    dd[ch] += 1 # 统计字符频率
```

Set(集合)

- 集合是一个无序、互不重复的可变集合
- 基本操作
 1. 创建：使用花括号 `{}` 定义，或者使用 `set()` 构造函数创建空集合，**注意空集合必须使用 `set()`，`{}` 是空字典**

```
s = {1, 2, 3} # 使用字面量创建集合
s = set([1, 2, 3]) # 使用可迭代对象创建集合
s = set() # 创建空集合
```

1. 集合中的元素必须是可哈希（不可变类型如数字、字符串、元组）
2. 添加：`s.add(x)` 添加元素，`s.update(iterable)` 添加多个元素
3. 删除：`s.remove(x)` 删除元素，不存在报错，`s.discard(x)` 删除元素，不存在不报错，`s.pop()` 随机删除一个元素并返回，`s.clear()` 清空集合
4. 查找：`x in s` 检查元素是否存在，`len(s)` 返回集合大小
5. 集合运算：支持数学上的集合运算，如交集 `&`、并集 `|`、差集 `-`、对称差集 `^` 等

函数

使用 `def` 关键字定义函数，函数可以有参数和返回值

参数与返回值

- 参数可以有默认值，支持可变数量的位置参数 `*args` 和关键字参数 `**kwargs`
- 函数体内可以使用 `return` 语句返回值（多个参数返回元组），默认返回 `None`

```
user_info("Alice", 25, "Beijing") # 位置传参
user_info(age=25, name="Alice", city="BJ") # 关键字传参，顺序无关
user_info("Alice", city="BJ", age=25) # 混合，位置参数必须在前
# *args: 可变位置参数 **kwargs: 可变关键字参数
func(1, 2, 3, a=4, b=5)
def func(*args, **kwargs):
    print("位置参数:", args) # args 是一个元组 (1, 2, 3)
    print("关键字参数:", kwargs) # kwargs 是一个字典 {'a': 4, 'b': 5}
```

:::tip

函数的默认参数是在 函数定义（Define）的那一刻被计算并存储的，而不是在 函数执行（Execute）的时候。这意味着如果默认参数是一个可变对象（如列表、字典等），并且在函数体内被修改了，那么这个修改会影响到所有使用该默认参数的函数调用，因为它们共享同一个对象。

```
def append_to(item, lst=[]):
    lst.append(item)
    return lst

append_to(1) # [1]
append_to(2) # [1, 2] ← 不是 [2]！共享同一个列表
```

...

作用域

- Python 中的变量作用域分为四类：局部作用域（Local）、嵌套作用域（Enclosing）、全局作用域（Global）和内置作用域（Built-in），简称 LEGB 规则。
- Python 按 $L \rightarrow E \rightarrow G \rightarrow B$ 顺序查找变量
- 使用 `global` 在函数内修改全局变量，使用 `nonlocal` 在嵌套函数内修改外层函数的变量

lambda 匿名函数

- lambda 表达式用于创建匿名函数，语法为 `lambda 参数: 表达式`，只能包含一个表达式，返回值是该表达式的结果

```
nums = [3, 1, 4, 1, 5, 9]
sorted(nums, key=lambda x: -x)          # 降序 [9,5,4,3,1,1]

people = [("Alice",25), ("Bob",20)]
sorted(people, key=lambda x: x[1])     # 按年龄排序
```

- 函数作为参数传递

```
# map: 对每个元素应用函数
list(map(lambda x: x**2, [1, 2, 3]))    # [1, 4, 9]
list(map(int, input().split()))        # 将输入的字符串分割并转换为整数列表

# filter: 过滤元素
list(filter(lambda x: x % 2 == 0, [1, 2, 3, 4])) # [2, 4]

# reduce: 累积计算
from functools import reduce
reduce(lambda x, y: x + y, [1, 2, 3, 4]) # 10
```

装饰器

- 装饰器是包裹函数的函数，在不修改原函数的情况下增加功能

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("函数开始执行")
        result = func(*args, **kwargs) # 调用原函数
        print("函数执行结束")
        return result
    return wrapper

# 使用装饰器
@my_decorator # 语法糖，等价于 say_hello = my_decorator(say_hello)
def say_hello(name):
    print(f"Hello, {name}!")
```

文件操作

- 打开与关闭

```
# 推荐: with 语句自动关闭
with open("data.txt", "r") as f:
```



```

content = f.read()

open(file, mode="r", encoding=None, errors=None)
# 常用模式
"r"      # 只读（默认），文件不存在报错
"w"      # 写入，文件不存在则创建，存在则清空
"a"      # 追加，文件不存在则创建，存在则在末尾追加
"x"      # 独占创建，文件已存在则报错

"rb"     # 二进制只读
"wb"     # 二进制写入
"ab"     # 二进制追加

"r+"     # 读写，文件必须存在
"w+"     # 读写，清空原内容
"a+"     # 读写，追加模式

```

- 读取与写入

```

with open("data.txt", "r+", encoding="utf-8") as f:
    content = f.read() # 读取整个文件内容为字符串
    lines = f.readlines() # 读取所有行，返回列表
    line = f.readline() # 读取一行，返回字符串
    for line in f: # 逐行读取文件
        print(line.strip()) # 去除行末的换行符

f.write("Hello, world!\n") # 写入字符串到文件
print("追加内容", file=f) # 使用 print 写入文件，自动添加换行符

# seek(offset, whence=0) 用于移动文件指针，offset 是偏移量，whence 指定参考位置（0=文件开头，1=当前位置，2=文件末尾）

```

面向对象

Python 是一种面向对象的编程语言，支持类和对象的概念。类是对象的蓝图或模板，对象是类的实例。

类与对象、属性与方法

- 定义类使用 `class` 关键字，类体内定义属性和方法
- 属性：类属性（所有实例共享）和实例属性（每个实例独有），通过 `self` 访问实例属性，通过类名访问类属性，实例也可以访问类属性但不能修改
:::tip Python 没有真正的私有属性
- 使用单下划线 `_` 前缀表示属性或方法是内部使用的（约定俗成）
- 使用双下划线 `__` 前缀会触发名称重整（name mangling），使得外部无法直接访问该属性或方法
:::

```

# 定义类
class Counter:
    count = 0 # 类属性，所有实例共享
    # 构造方法，每次创建实例时调用
    def __init__(self):
        Counter.count += 1
        self.id = Counter.count # 实例属性，每个独有

```

```

a = Counter()    # count=1, a.id=1
b = Counter()    # count=2, b.id=2
c = Counter()    # count=3, c.id=3

Counter.count    # 3 （类读类属性）
a.count          # 3 （实例读类属性）

# 实例赋值会遮蔽类属性，不会修改类属性
a.count = 99     # 创建实例属性，类属性不变
Counter.count    # 仍然是 3

```

- 方法：定义在类中的函数，
- 实例方法：第一个参数通常是 `self`，表示实例本身，可以访问实例属性和方法
- 类方法：使用 `@classmethod` 装饰器，第一个参数是 `cls`，表示类本身，可以访问类属性和方法
- 静态方法：使用 `@staticmethod` 装饰器，不依赖实例或类，通常用于工具函数

```

class MyClass:
    class_var = "共享"

    def __init__(self, value):
        self.value = value

    # 实例方法：第一个参数是 self（实例本身）
    def instance_method(self):
        return f"实例方法, value={self.value}"

    # 类方法：第一个参数是 cls（类本身）
    @classmethod
    def class_method(cls):
        return f"类方法, class_var={cls.class_var}"

    # 静态方法：不依赖实例或类
    @staticmethod
    def static_method(x, y):
        return x + y

obj = MyClass(42)

obj.instance_method()    # '实例方法, value=42'
MyClass.class_method()   # '类方法, class_var=共享'
obj.class_method()       # 实例也可调用类方法
MyClass.static_method(3,4) # 7
obj.static_method(3, 4)  # 7

```

继承与多态

- 继承：子类可以继承父类的属性和方法，使用 `class SubClass(ParentClass):` 定义子类，子类可以重写父类的方法，也可以调用父类的方法
- 使用 `super()` 函数调用父类的方法，特别是在重写方法时需要调用父类的版本来保持原有功能
- 使用 `isinstance()` 和 `issubclass()` 函数检查对象和类的关系，支持多态性，即子类对象可以被当作父类对象使用

```

class Animal:

```

```

def __init__(self, name, age):
    self.name = name
    self.age = age

def eat(self):
    return f"{self.name} is eating"

def speak(self):
    raise NotImplementedError("子类必须实现 speak()")

def __str__(self):
    return f"{self.__class__.__name__}(name={self.name})"

# 子类
class Dog(Animal):
    def __init__(self, name, age, breed):
        super().__init__(name, age)    # 调用父类 __init__
        self.breed = breed

    def speak(self):                  # 重写父类方法
        return f"{self.name} says: woof!"

    def fetch(self):                  # 子类独有方法
        return f"{self.name} fetches the ball!"

dog = Dog("Rex", 3, "Labrador")
dog.eat()        # 'Rex is eating'    ← 继承自 Animal
dog.speak()      # 'Rex says: woof!' ← Dog 自己的
dog.fetch()      # 'Rex fetches the ball!'

# isinstance 和 isinstance
isinstance(dog, Dog)        # True
isinstance(dog, Animal)    # True, Dog 是 Animal 的子类
isinstance(Dog, Animal)    # True

```

- 多态：不同类的对象可以通过相同的接口调用方法，表现出不同的行为

```

# 同一接口，不同实现
class Shape:
    def area(self):
        raise NotImplementedError

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r ** 2

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w = w
        self.h = h
    def area(self):
        return self.w * self.h

```

```

class Triangle(Shape):
    def __init__(self, b, h):
        self.b = b
        self.h = h
    def area(self):
        return 0.5 * self.b * self.h

# 多态: 统一处理不同类型
shapes = [Circle(5), Rectangle(4,6), Triangle(3,8)]
for s in shapes:
    print(f"{s.__class__.__name__}: {s.area():.2f}")

# 鸭子类型: 不需要继承关系, 有相同方法即可
class Star:
    def area(self):
        return 42

# Star 不继承 Shape, 但有 area() 方法, 一样可用
shapes.append(Star())

```

魔术方法

- 魔术方法 (Magic Methods) 是以双下划线 `__` 开头和结尾的方法, 也称为特殊方法 (Special Methods), 用于实现类的特殊行为
- 通过定义魔术方法, 可以让自定义类的对象表现得像内置类型一样, 支持运算符重载、迭代、上下文管理等功能
- 常见的魔术方法包括:

```

# 生命周期
__init__      # 初始化
__del__       # 析构 (对象被回收时)
__new__       # 创建实例 (单例模式用)

# 字符串
__str__       # str(obj), print(obj)
__repr__      # repr(obj), 交互式显示
__format__    # format(obj, spec)

# 运算符
__add__       # +
__sub__       # -
__mul__       # *
__truediv__   # /
__mod__       # %
__floordiv__  # //
__pow__       # **

# 比较
__eq__        # ==   __ne__  # !=
__lt__        # <    __le__  # <=
__gt__        # >    __ge__  # >=

# 容器

```

```
__len__      # len()
__getitem__  # obj[key]
__setitem__  # obj[key] = val
__delitem__  # del obj[key]
__contains__ # in 运算符
__iter__     # for 循环
__next__     # next()

# 上下文管理器
__enter__    # with 进入
__exit__     # with 退出
```